

Trabalho Prático 1

Controlador de Cache

Implementação em SystemVerilog

Computer Organization and Design: RISC-V Edition
Capítulo 5, Seção 5.12

Organização e Arquitetura de Computadores

1. Introdução

A hierarquia de memória é um dos pilares fundamentais da arquitetura de computadores modernos. Caches existem para mitigar a disparidade de velocidade entre processador e memória principal, mantendo localmente cópias dos dados acessados com maior frequência. O controlador de cache é o componente de hardware responsável por toda a lógica de acesso, substituição e consistência entre esses níveis da hierarquia.

Este relatório descreve a implementação de um controlador de cache **direct-mapped** com política **write-back** e **write-allocate** em SystemVerilog, conforme especificado no Capítulo 5, Seção 5.12 do livro *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition* (Patterson & Hennessy).

Os objetivos do trabalho são: consolidar o entendimento sobre funcionamento de caches, implementar em nível de hardware os mecanismos de controle, validar funcionalmente o projeto por meio de simulação automatizada com 26 testes, e documentar as decisões de projeto adotadas.

2. Descrição do Desenvolvimento

2.1 Arquitetura Geral

O sistema é composto por dois módulos SystemVerilog. O **cache_controller** é o dispositivo sob teste (DUT) e implementa toda a lógica de controle da cache. O **main_memory** é um modelo comportamental de memória principal com latência configurável, utilizado exclusivamente nas simulações.

A interface do controlador com a CPU usa sinais de *handshake* (**cpu_req/cpu_ack/cpu_stall**): a CPU apresenta uma requisição e aguarda, sendo travada (stall) enquanto a cache trata um miss. A interface com a memória usa protocolo burst de 4 palavras para carga e descarga de blocos completos.

Módulo	Função	Arquivo
cache_controller	FSM principal: hit/miss, write-back, alocação, handshake	rtl/cache_controller.sv
main_memory	Modelo de memória com latência configurável (2 ciclos)	rtl/main_memory.sv
tb_cache_controller	Testbench automatizado com 26 testes em 6 suites	tb/tb_cache_controller.sv

Tabela 1 — Módulos do projeto

2.2 Parâmetros e Decomposição do Endereço

A cache é parametrizada e configurada por padrão com 1 KB de capacidade, organizada em 64 blocos de 16 bytes. Com mapeamento direto, o endereço de 32 bits é decomposto em três campos:

Campo	Bits	Largura	Descrição
Tag	[31:10]	22 bits	Identifica qual bloco da memória ocupa a linha

Campo	Bits	Largura	Descrição
Index	[9:4]	6 bits	Seleciona qual das 64 linhas da cache acessar
Offset	[3:0]	4 bits	Byte dentro do bloco (bits [3:2] = offset de palavra)

Tabela 2 — Decomposição do endereço de 32 bits

2.3 Estrutura de Armazenamento

Cada linha da cache armazena quatro campos: **valid** (1 bit), **dirty** (1 bit), **tag** (22 bits) e **data** (4 × 32 bits = 128 bits). Os campos foram declarados como arrays independentes — e não como *packed struct* — para garantir compatibilidade com Icarus Verilog, ModelSim, Questa e Verilator:

```
logic          cache_valid [0:63];    // bit de validade
logic          cache_dirty [0:63];    // bit dirty (write-back)
logic [21:0]    cache_tag   [0:63];    // tag de 22 bits
logic [31:0]    cache_data  [0:63][0:3]; // 4 palavras por bloco
```

2.4 Máquina de Estados Finitos (FSM)

O controlador é implementado como uma FSM síncrona de 4 estados com reset assíncrono ativo em nível baixo. A lógica de próximo estado é combinacional (always_comb) e os registradores são atualizados na borda de subida do clock:

Estado	Função
IDLE	Aguarda requisição da CPU. Ao receber cpu_req=1, captura endereço, dado e tipo de operação em registradores, e a
COMPARE_TAG	Verifica hit: valid=1 E tag==saved_tag. Em hit, serve a CPU (1 ciclo) e volta ao IDLE. Em miss, decide entre WRITE_B
WRITE_BACK	Escreve as 4 palavras do bloco dirty na memória via burst. Usa burst_cnt (2 bits) para sequenciar. Ao concluir, transit
ALLOCATE	Carrega as 4 palavras do novo bloco da memória via burst. Ao concluir, atualiza valid, dirty=0 e tag, e retorna ao COM

Tabela 3 — Estados da FSM

```
IDLE      → cpu_req=1      → COMPARE_TAG
COMPARE_TAG → hit          → IDLE
COMPARE_TAG → miss, dirty=0 → ALLOCATE
COMPARE_TAG → miss, dirty=1 → WRITE_BACK
WRITE_BACK → mem_ack, burst_cnt==3 → ALLOCATE
ALLOCATE   → mem_ack, burst_cnt==3 → COMPARE_TAG
```

2.5 Políticas Adotadas

Política	Escolha	Justificativa
Organização	Direct-Mapped	Especificada no livro (Seção 5.12). Hardware mínimo, sem comparadores de via.
Escrita (hit)	Write-Back	Atualiza apenas a cache e marca dirty=1. Reduz tráfego com a memória principal.
Escrita (miss)	Write-Allocate	Aloca o bloco antes de escrever. Favorece localidade espacial.
Substituição	Determinística	Direct-mapped: o índice determina univocamente a linha. Sem política LRU/FIFO.

Política	Escolha	Justificativa
Burst de memória	4 palavras	Tamanho exato do bloco. burst_cnt de 2 bits sequencia as transferências.
Latência memória	2 ciclos	Configurável via parâmetro LATENCY no modelo main_memory.sv.

Tabela 4 — Decisões de projeto

2.6 Fluxo de Write-Back e Write-Allocate

Em um **miss de leitura** com bloco dirty, o controlador reconstrói o endereço de destino usando o tag *armazenado* na linha (não o tag da nova requisição): `wb_base = {cache_tag[idx], idx, 4'b0}`. O write-back emite 4 requisições de escrita sequenciais à memória (burst). Somente após a conclusão do write-back a alocação do novo bloco é iniciada.

Em uma **escrita com hit**, apenas a palavra endereçada é atualizada na cache e dirty é setado. A memória não é acessada. Em uma **escrita com miss** (write-allocate), o bloco é primeiro carregado (ALLOCATE), e após retornar ao COMPARE_TAG o hit é confirmado e a escrita é executada normalmente.

2.7 Dificuldades Encontradas

Compatibilidade de simuladores: A versão inicial utilizava *packed structs* com arrays para representar as linhas de cache, construção não suportada pelo Icarus Verilog 12.0. A solução foi separar os campos em quatro arrays independentes, tornando o código portátil para todos os simuladores principais.

Timing de sinais registrados no testbench: O sinal `cpu_stall` é registrado (atualizado na borda do clock), então só fica estável dois ciclos após a requisição. O testbench inicial amostrava após um único ciclo, capturando o valor de transição. A solução foi aguardar dois ciclos antes de amostrar o stall.

Endereço de write-back: O endereço de destino durante o write-back deve usar o tag *antigo* (salvo na linha de cache), não o tag da requisição corrente. Uma confusão entre esses dois valores resultaria em corrupção de dados na memória. O sinal `wb_base` foi declarado como assign combinacional sobre `cache_tag[saved_index]` para evitar esse erro.

3. Resultados Obtidos

3.1 Resultados da Simulação

Todos os 26 testes automatizados passaram com sucesso na simulação com Icarus Verilog 12.0. Os testes cobrem todos os cenários obrigatórios da Seção 7 do enunciado:

Suite	Descrição	Testes	Status
1 — Read Path	Hit, miss, carregamento de bloco, valid/tag após alocação	5	✓ PASS
2 — Write Path	Write hit, write miss, dirty bit, política write-back verificada	4	✓ PASS
3 — Substituição	Write-back de bloco dirty, preenchimento de 64 blocos, direct-mapped	6	✓ PASS
4 — Consistência	R→W→R, 10 leituras repetidas, conflito de índice com write-back	3	✓ PASS
5 — Casos Limite	Addr=0x0, addr_max=0x3FFC, reset completo, estado vazio	5	✓ PASS
6 — Handshake	cpu_stall durante miss, cpu_ack na conclusão, liberação de stall	3	✓ PASS
Total		26	26/26

Tabela 5 — Resultados da simulação (26/26 PASS)

3.2 Log Completo da Simulação

Saída integral do testbench executado com `vvp sim/cache_sim`:

```
=====
TESTBENCH – Controlador de Cache
1KB, 64 blocos, direct-mapped, write-back, write-alloc
=====

--- Suite 1: Read Path ---
[PASS] 1.1 Read Miss retorna dado correto da memória
[PASS] 1.2 Read Hit retorna mesmo valor
[PASS] 1.3 Read Hit outra palavra do bloco
[PASS] 1.4 valid=1 após miss+alocação
[PASS] 1.5 Tag correta após alocação

--- Suite 2: Write Path ---
[PASS] 2.1 Write Hit – cache atualizada
[PASS] 2.2 Write Hit – dirty=1
[PASS] 2.3 Write-Back – mem não atualizada imediatamente
[PASS] 2.4 Write Miss (write-allocate) – leitura retorna valor escrito

--- Suite 3: Substituição e Write-Back ---
[PASS] 3.1 Write-Back ocorre ao substituir bloco dirty
[PASS] 3.2 Bloco substituto valid=1
[PASS] 3.2 Tag do bloco substituto correta
Preenchendo todos os 64 blocos...
[PASS] 3.3 Todos os 64 blocos preenchidos (valid=1)
[PASS] 3.4 Direct-Mapped: acesso conflitante substitui bloco anterior
[PASS] 3.4 Direct-Mapped: bloco substituído não está mais na cache (tag mudou)

--- Suite 4: Consistência ---
```

```
[PASS] 4.1 R→W→R: leitura reflete escrita
[PASS] 4.2 Valor estável em 10 leituras consecutivas
[PASS] 4.3 Conflito de índice – write-back preservou dado na mem
```

```
--- Suite 5: Casos Limite ---
```

```
[PASS] 5.1 Acesso ao endereço 0x0
[PASS] 5.2 Acesso ao endereço extremo alto (0x3FFC)
[PASS] 5.3 Reset – todos valid=0
[PASS] 5.4 Reset – todos dirty=0
[PASS] 5.5 Primeiro acesso após reset carrega dado correto
```

```
--- Suite 6: Handshake CPU ---
```

```
[PASS] 6.1 cpu_stall=1 durante miss
[PASS] 6.2 cpu_ack=1 ao concluir transação
[PASS] 6.3 cpu_stall=0 após conclusão
```

```
=====
RESULTADO FINAL
Total : 26 | Passou : 26 | Falhou : 0
*** TODOS OS TESTES PASSARAM ***
=====
```

3.3 Análise de Waveforms

O testbench gera automaticamente o arquivo `sim/cache_sim.vcd`, que pode ser inspecionado no GTKWave com o comando `gtkwave sim/cache_sim.vcd`. A tabela abaixo descreve o comportamento esperado nos principais sinais para cada tipo de transação, servindo como guia de análise do waveform:

Transação	cpu_req	cpu_stall	cpu_ack	mem_req	Ciclos
Read Hit	1→0	1→0	pulso 1	0	~2
Read Miss (limpo)	1→0	1→0	pulso 1	burst 4	~11
Read Miss (dirty)	1→0	1→0	pulso 1	burst 8	~19
Write Hit	1→0	1→0	pulso 1	0	~2
Write Miss	1→0	1→0	pulso 1	burst 4	~11

Tabela 6 — Comportamento de sinais por tipo de transação (latência mem = 2 ciclos)

3.4 Análise de Desempenho

Com latência de memória de 2 ciclos e burst de 4 palavras, o custo de cada operação é:

Operação	Condição	Ciclos totais
Leitura	Hit	2 (IDLE + COMPARE_TAG)
Leitura	Miss — bloco limpo	2 + 4×2 = 10 (alocação) + 1 serve = 11
Leitura	Miss — bloco dirty	2 + 4×2 (WB) + 4×2 (aloc) + 1 = 19
Escrita	Hit	2

Operação	Condição	Ciclos totais
Escrita	Miss — write-allocate, limpo	11
Escrita	Miss — write-allocate, dirty	19

Tabela 7 — Latência por operação

4. Conclusão

O projeto atingiu integralmente os objetivos propostos. O controlador de cache foi implementado em SystemVerilog com arquitetura direct-mapped, política write-back e write-allocate, interface de handshake com CPU e protocolo burst com a memória, seguindo fielmente a especificação do livro.

A FSM de 4 estados demonstrou-se uma solução clara, modular e robusta. Os 26 testes automatizados — organizados em 6 suites que cobrem exatamente os cenários obrigatórios da Seção 7 do enunciado — passaram integralmente na simulação com Icarus Verilog 12.0, e o waveform VCD gerado permite inspeção visual de qualquer transação.

As dificuldades encontradas foram de natureza prática (compatibilidade de simuladores e timing de sinais registrados) e foram resolvidas e documentadas de forma sistemática, reforçando a compreensão dos mecanismos envolvidos.

Como extensão natural, seria interessante implementar uma cache **set-associative de 2 ou 4 vias** com política LRU, permitindo comparação quantitativa de taxa de miss entre as organizações para diferentes padrões de acesso.

5. Uso de Inteligência Artificial

O desenvolvimento deste trabalho utilizou o modelo **Claude Sonnet 4.6** (Anthropic, acessado via claude.ai) como assistente de desenvolvimento, seguindo o modelo **Spec-Driven Development (SDD)** exigido no enunciado.

5.1 Prompts Utilizados

Foram utilizados três prompts ao longo da sessão de desenvolvimento. Os dois primeiros foram continuações da mesma conversa (a interface foi interrompida entre as fases de implementação):

Prompt 1 — Abertura e especificação do trabalho

Preciso que atue como um desenvolvedor profissional e faça esse trabalho com extrema competência [PDF do enunciado anexado ao prompt]

Contexto: O PDF completo do enunciado foi anexado diretamente no prompt. Esse prompt definiu o escopo completo: implementar o controlador conforme a Seção 5.12 do livro, com testbench cobrindo todos os cenários da Seção 7, relatório com seção de IA, e README para repositório GitHub.

Prompt 2 — Retomada após interrupção (fase RTL + TB)

continue

Contexto: A geração foi interrompida durante a criação dos arquivos RTL. Este prompt retomou o processo a partir do ponto de parada, completando o `cache_controller.sv` revisado (sem packed structs), o testbench corrigido (timing do stall) e o script de simulação.

Prompt 3 — Revisão completa e correções finais

quero que revise tudo para ter certeza que tudo está certo e que está da forma correta de acordo com o que foi pedido no trabalho e se contem tudo necessário

Contexto: Este prompt gerou uma revisão sistemática contra o enunciado, identificando e corrigindo 6 problemas: remoção do registrador `saved_addr` não utilizado, adição de 2 testes de substituição `direct-mapped`, correção do script `run_sim.sh` para execução de qualquer diretório, adição do `.gitignore`, e atualização do relatório com log e contagens corretas.

5.2 Como a IA foi Utilizada

Etapas	Uso da IA	Responsabilidade do grupo
Arquitetura	Sugestão da FSM de 4 estados e decomposição de <code>cache_controller.sv</code>	Validação contra o livro (Seção 5.12)
RTL	Geração do <code>cache_controller.sv</code> e <code>main_memory.sv</code>	Análise linha a linha; correção de bugs de compatibilidade
Testbench	Geração das 6 suites de teste (26 cenários)	Verificação de cobertura contra Seção 7; ajuste de timing
Depuração	Identificação da causa dos FAILs nas suites 3 e 6	Entendimento da FSM registrada; correção do TB
Revisão	Checklist sistemático contra o enunciado	Priorização e aprovação das correções a aplicar

Etapa	Uso da IA	Responsabilidade do grupo
Documentação	Redação do relatório, tabelas e README	Revisão técnica e verificação de acurácia de todos os dados

Tabela 8 — Detalhamento do uso de IA por etapa

5.3 Responsabilidade Técnica

O grupo revisou e compreende integralmente todo o código entregue. A IA foi utilizada como acelerador de desenvolvimento e ferramenta de depuração, nunca como substituta do entendimento técnico. O grupo é capaz de explicar e defender cada decisão implementada, incluindo: a decomposição tag/index/offset, a lógica de transição da FSM, a reconstrução do endereço de write-back, o protocolo de burst, e a diferença prática entre write-back e write-through.